Algorithms 3

COMP2711 Discrete Mathematical Tools

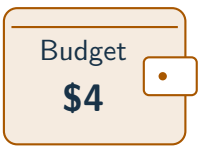
Mingxun Zhou (mingxunz@ust.hk)

Which treasures should we buy?









Problem

Maximize total value with total price ≤ 4.
No splitting or repeats.

Which treasures should we buy?



Problem

Maximize total value with total price ≤ 4.
No splitting or repeats.

B + C: price $2 + 2 = 4$, value $3 + 3 = 6$.

Lecture roadmap

1. **Brute force**
   Try every subset.

2. **Greedy and approximation**
   Choose quickly; guarantee quality.

3. **P, NP, and NP-completeness**
   Connect knapsack with Boolean formulas.

SECTION 1

Brute force

The 0/1 knapsack problem

Item i: price p_i dollars, value v_i points (positive integers). Budget $B \geq 0$.

Optimization problem: choose a subset S.

Treasure	Price	Value
 A	3	5
 **B**	2	3
 **C**	2	3

Selected subset S

Constraint

$$\sum_{i \in S} p_i \leq B$$

Objective: maximize value

$$\max_{S} \sum_{i \in S} v_i$$

Brute force

MUST UNDERSTAND **Brute force**

Try every subset: 2^n **candidates**. Pick the best that fits.

Subset	Price	Value	Fits $B = 4$?
$\varnothing$	0	0	Yes
A	3	5	Yes
B	2	3	Yes
C	2	3	Yes
A,B	5	8	No
A,C	5	8	No
B,C	4	6	Yes
A,B,C	7	11	No

Brute force

MUST UNDERSTAND **Brute force**

Try every subset: 2^n **candidates**. Pick the best that fits.

Subset	Price	Value	Fits $B = 4$?
$\varnothing$	0	0	Yes
A	3	5	Yes
B	2	3	Yes
C	2	3	Yes
A,B	5	8	No
A,C	5	8	No
B,C	4	**6**	Yes
A,B,C	7	11	No

Best: **B + C**, value **6**.

SECTION 2

Greedy algorithms and approximation

Alice: Highest value first

MUST UNDERSTAND **Greedy algorithm**

Choose the best by a **local rule**; keep that choice.

Alice's strategy

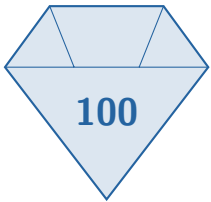
Take items in decreasing value order.

Skip any item you cannot afford.



Highest value first can fail badly

Budget: 100.



Price: 100



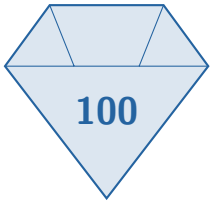
10 treasures; price 10 each

Alice: the large treasure

Value: **100**. The budget is spent.

Highest value first can fail badly

Budget: 100.



Price: 100



10 treasures; price 10 each

Alice: the large treasure

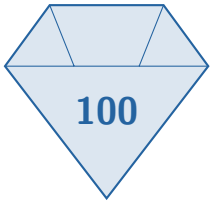
Value: **100**. The budget is spent.

Optimal: all small treasures

Value: $10 \times 99 = \mathbf{990}$. Price: 100.

Highest value first can fail badly

Budget: $10k$, where $k \geq 2$ is an integer.



Price: $10k$



$\times\ k$

k treasures; price 10 each

Alice: the large treasure

Value: **100**. The budget is spent.

Optimal: all small treasures

Value: **$99k$**. Price: $10k$.

$$\frac{\text{Alice's value}}{\text{Best}} = \frac{100}{99k} \longrightarrow 0.$$

Arbitrarily bad.

Bob: Highest value per dollar

Bob's strategy

Sort by decreasing **value per dollar**: v_i/p_i.
性价比

Buy each item if the remaining budget allows; otherwise, skip it.



Bob's strategy: Picking in sorted order

Budget: 5.

Item	Price	Value	Value per dollar	Choice
D	1	2	2	**Take**
A	3	5	≈ 1.67	
B	2	3	1.5	
C	2	3	1.5	

Remaining budget

$5 - 1 = \mathbf{4}$

Take D.

Value: **2**.

Bob's strategy: Picking in sorted order

Budget: 5.

Item	Price	Value	Value per dollar	Choice
D	1	2	2	**Take**
A	3	5	≈ 1.67	**Take**
B	2	3	1.5	
C	2	3	1.5	

Remaining budget

$4 - 3 = \mathbf{1}$

Take A.

Value: $2 + 5 = \mathbf{7}$.

Bob's strategy: Picking in sorted order

Budget: 5.

Item	Price	Value	Value per dollar	Choice
D	1	2	2	**Take**
A	3	5	≈ 1.67	**Take**
B	2	3	1.5	Skip
C	2	3	1.5	

Remaining budget

1

B needs 2: skip.

Value: $2 + 5 = \mathbf{7}$.

Bob's strategy: Picking in sorted order

Budget: 5.

Item	Price	Value	Value per dollar	Choice
D	1	2	2	**Take**
A	3	5	≈ 1.67	**Take**
B	2	3	1.5	Skip
C	2	3	1.5	Skip

Remaining budget

1

C needs 2: skip.

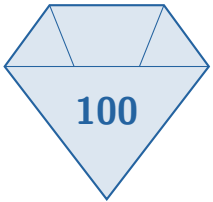
Value: $2 + 5 = \mathbf{7}$.

Bob's strategy can fail badly, too

Budget: 100.



Price: 1
Value per dollar: 2



Price: 100
Value per dollar: 1

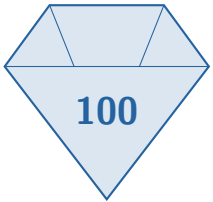
Bob's strategy can fail badly, too

Budget: 100.



Price: 1
Value per dollar: 2

Bob picks this first.



Price: 100
Value per dollar: 1

Only 99 dollars left: too expensive.

捡了芝麻，丢了西瓜

"Pick up a sesame seed, but lose a watermelon."

Carol: Combine Alice's and Bob's ideas



Alice
Highest value first



Bob
Value per dollar



Carol
Keep the better selection.

How good is Carol's strategy?

Alice and Bob can each be **arbitrarily bad**.

What about Carol?

How good is her selection in the **worst case**?

How good is Carol's strategy?

Alice and Bob can each be **arbitrarily bad**.

What about Carol?

How good is her selection in the **worst case**?

Surprisingly: at least half the optimum!

$$\text{Carol's value} \geq \frac{1}{2} \text{ optimal selection value.}$$

The approximation guarantee

Best: best achievable value. $\mathrm{Value}^{\mathrm{Carol}}$: returned value.

MUST UNDERSTAND **Approximation guarantee**

Carol is an α-approximation if her selection satisfies

$$\mathrm{Value}^{\mathrm{Carol}} \geq \alpha\,\mathrm{Best} \quad \text{on every input.}$$

The approximation guarantee

Best: best achievable value. $\mathrm{Value}^{\mathrm{Carol}}$: returned value.

MUST UNDERSTAND **Approximation guarantee**

Carol is an α-approximation if her selection satisfies

$$\mathrm{Value}^{\mathrm{Carol}} \geq \alpha\,\mathrm{Best} \quad \text{on every input.}$$

Theorem: Carol's strategy

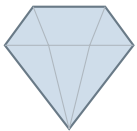
$$\tfrac{1}{2}\mathbf{Best} \leq \mathbf{Value}^{\mathbf{Carol}} \leq \mathbf{Best}.$$

At least **half** the optimal value.

An upper bound: Suppose we allow fractions

Allow fractions to get an **upper bound**.

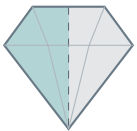
All of A



Price: 3 Value: 5

$+$

Half of B

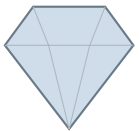
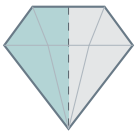


Price: 1 Value: 1.5

Total price: $3 + 1 = 4$.

An upper bound: Suppose we allow fractions

Allow fractions to get an **upper bound**.

All of A  + **Half of B** 

Price: 3 Value: 5 Price: 1 Value: 1.5

Total price: $3 + 1 = 4$.

More flexibility cannot lower the optimum

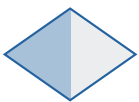
$$\text{Best} \leq \text{Best}^{\text{Fractional}}.$$

Here: $\text{Best}^{\text{Fractional}} = 5 + 1.5 = 6.5$.

Allowing fractions: Fact 1
Value per dollar is the best strategy

$r_A \leq r_B$

Before: A



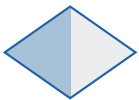
Spent d dollars on A



Allowing fractions: Fact 1
Value per dollar is the best strategy

$r_A \leq r_B$

Before: A



Spent d dollars on A

Same spending

After: B



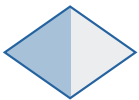
Spending d dollars on B



Allowing fractions: Fact 1
Value per dollar is the best strategy

$r_A \leq r_B$

Before: A



Spent d dollars on A

Same spending

After: B



Spending d dollars on B

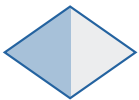


$$\text{Value}_{\text{after}} - \text{Value}_{\text{before}} = (r_B - r_A)d \geq 0.$$

Allowing fractions: Fact 1
Value per dollar is the best strategy

$r_A \leq r_B$

Before: A



Spent d dollars on A

Same spending

After: B



Spending d dollars on B



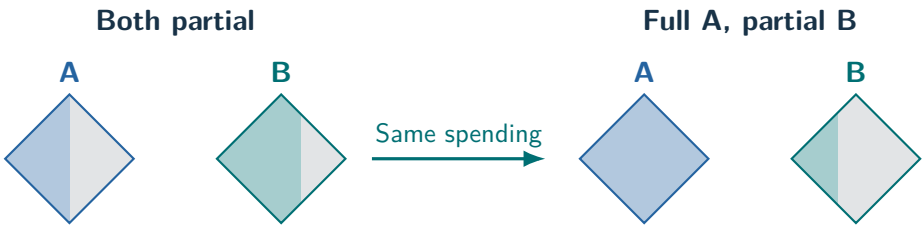
$$\text{Value}_{\text{after}} - \text{Value}_{\text{before}} = (r_B - r_A)d \geq 0.$$

Therefore

In a best fractional solution, never buy a strictly lower-ratio item while a higher-ratio item is not fully bought.

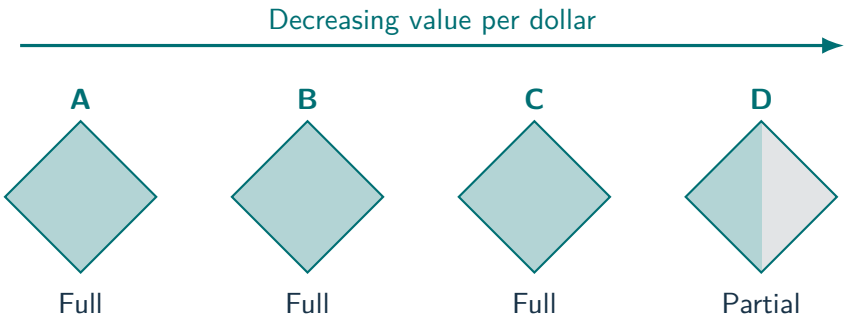
Allowing fractions: Fact 2
At most one fractional item is needed

Assume value per dollar: $r_A \geq r_B$.



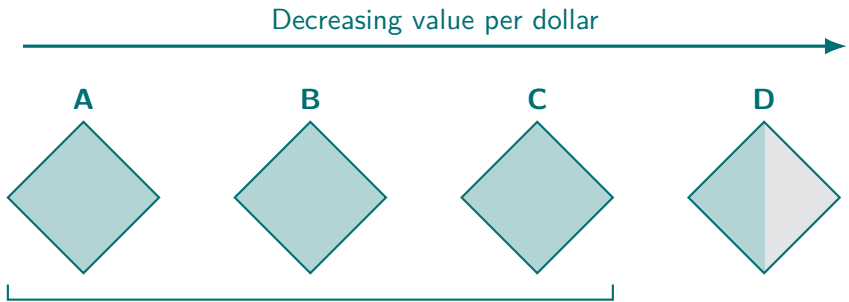
The fractional optimum

Allowing fractions, there exists an optimal selection of this form:



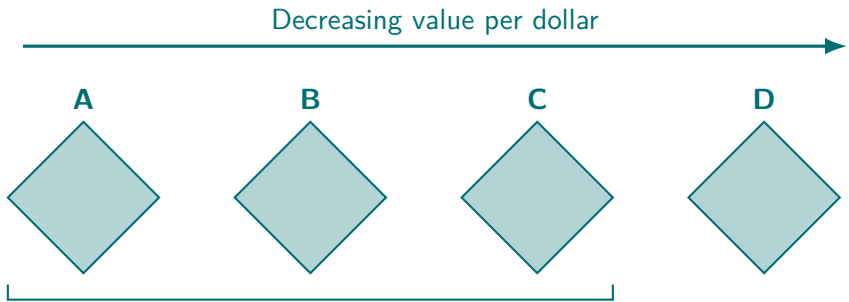
An upper bound on the fractional optimum

$$\text{Best} \leq \text{Best}^{\text{Fractional}} \leq \text{Value}^{\text{Bob}} + v_D \quad \text{(may exceed the budget).}$$



An upper bound on the fractional optimum

$$\text{Best} \leq \text{Best}^{\text{Fractional}} \leq \text{Value}^{\text{Bob}} + v_D \quad \text{(may exceed the budget).}$$



Recall Alice's greedy strategy

$$\mathrm{Best} \leq \mathrm{Best}^{\mathrm{Fractional}} \leq \mathrm{Value}^{\mathrm{Bob}} + v_D \quad \text{(may exceed the budget).}$$

Alice picks the **highest-value item that fits** first.

Her selection is worth at least as much as **any single item that fits**.



Recall Alice's greedy strategy

$$\mathrm{Best} \leq \mathrm{Best}^{\mathrm{Fractional}} \leq \mathrm{Value}^{\mathrm{Bob}} + v_D \quad \text{(may exceed the budget).}$$

Alice picks the **highest-value item that fits** first.

Her selection is worth at least as much as **any single item that fits**.



D fits by itself

$$v_D \leq \mathrm{Value}^{\mathrm{Alice}}.$$

Why Carol gets at least half

Combine the bounds

$$\text{Best} \leq \text{Best}^{\text{Fractional}} \leq \text{Value}^{\text{Bob}} + v_D \leq \text{Value}^{\text{Bob}} + \text{Value}^{\text{Alice}}.$$

Why Carol gets at least half

Combine the bounds

$$\text{Best} \leq \text{Best}^{\text{Fractional}} \leq \text{Value}^{\text{Bob}} + v_D \leq \text{Value}^{\text{Bob}} + \text{Value}^{\text{Alice}}.$$

Carol keeps the better selection

$$\begin{aligned}\text{Value}^{\text{Carol}} &= \max(\text{Value}^{\text{Alice}}, \text{Value}^{\text{Bob}}) \\ &\geq \frac{\text{Value}^{\text{Alice}} + \text{Value}^{\text{Bob}}}{2} \geq \frac{\textbf{Best}}{\textbf{2}}.\end{aligned}$$

Approximation algorithms

A useful tradeoff

Fast solutions with **guaranteed quality**.

Our knapsack algorithm

$O(n \log n)$ comparisons.

At least $\frac{1}{2}\mathrm{Best}$.

Research questions

Better quality?

At what time cost?

Approximation is a major area of algorithms research.

Beyond this lecture

Greedy algorithms

A major area of algorithm research.

Exact knapsack: dynamic programming

A standard method for finding the optimal selection.

You will learn it in your algorithm class.

SECTION 3

P, NP, and NP-completeness

Knapsack: Optimization and decision

n items with prices $p_1, \ldots, p_n$ and values $v_1, \ldots, v_n$.
Budget: B.

Optimization version

Find a subset S maximizing $\sum_{i \in S} v_i$ subject to $\sum_{i \in S} p_i \leq B$.

Output: **an optimal selection**.

Decision version

Given a target T, is there a subset S with

$$\sum_{i \in S} p_i \leq B \qquad \text{and} \qquad \sum_{i \in S} v_i \geq T \ ?$$

Output: **Yes or No**.

Checking a proposed selection

Decision version

Given a target T, is there a subset S with

$$\sum_{i \in S} p_i \leq B \qquad \text{and} \qquad \sum_{i \in S} v_i \geq T \ ?$$

Output: **Yes or No**.

Proposed selection: **items 2, 3, and 8**.

$$p_2 + p_3 + p_8 \leq B \qquad \text{and} \qquad v_2 + v_3 + v_8 \geq T \ ?$$

Easy to check a proposal: $O(n)$ time.

Find a proposal?

Checking a solution vs. finding a solution

Checking a solution

Given a proposed solution, decide whether it is valid.

Finding a solution

Produce a valid solution, or determine that none exists.

Checking a solution vs. finding a solution

Checking a solution

Given a proposed solution, decide whether it is valid.

Finding a solution

Produce a valid solution, or determine that none exists.

Can finding be strictly harder than checking?

Does fast checking imply fast finding?

Polynomial time

MUST UNDERSTAND **Polynomial time**

Worst-case running time $O(n^c)$, where n is the input size and c is a fixed constant.

Polynomial	Exponential
n, $n \log n$, n^2, n^3.	2^n.

P and NP

Classes of **decision problems**.

MUST UNDERSTAND **P**

The Yes/No answer can be **found** in polynomial time.

P and NP

Classes of **decision problems**.

MUST UNDERSTAND **P**

The Yes/No answer can be **found** in polynomial time.

MUST UNDERSTAND **NP**

A Yes solution can be **checked** in polynomial time.

No answer: no valid certificate.

P and NP

Classes of **decision problems**.

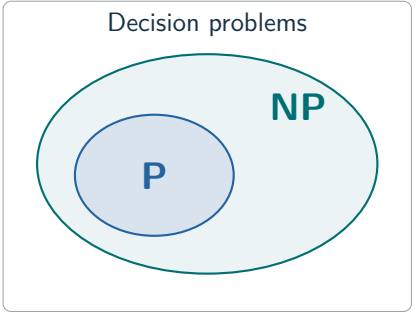
MUST UNDERSTAND P

The Yes/No answer can be **found** in polynomial time.

MUST UNDERSTAND NP

A Yes solution can be **checked** in polynomial time.

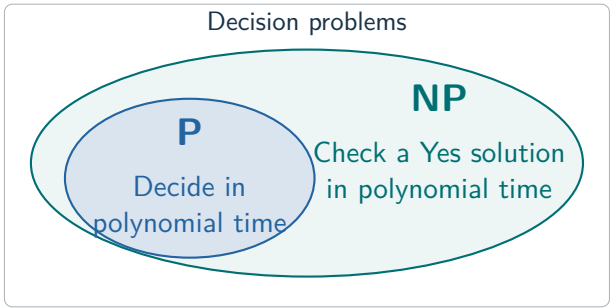
No answer: no valid certificate.



$\mathrm{P} \subseteq \mathrm{NP}$

P versus NP: A holy grail of computer science

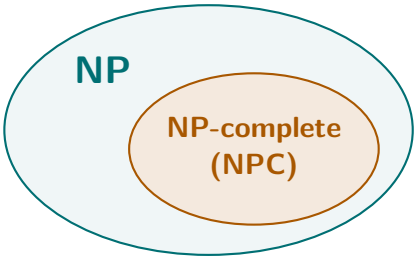
$$\mathrm{P} \stackrel{?}{=} \mathrm{NP}$$





Is there a problem where finding a solution is **fundamentally harder** than checking it?

NP-complete: The hardest problems in NP

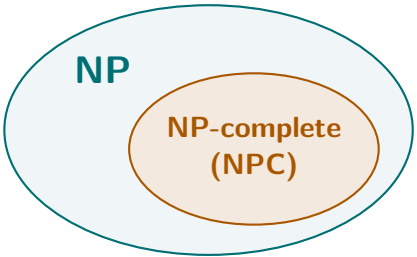


Why special?

Every NP problem can be converted to an NP-complete problem in polynomial time.

A $T(n)$-time algorithm for an NP-complete problem gives an algorithm for any NP problem in $p(n) + T(q(n))$ time, for polynomials p, q.

NP-complete: The hardest problems in NP



Why special?

Every NP problem can be converted to an NP-complete problem in polynomial time.

A $T(n)$-time algorithm for an NP-complete problem gives an algorithm for any NP problem in $p(n) + T(q(n))$ time, for polynomials p, q.

Surprisingly

Decision knapsack is NP-complete.

Appendix

3SAT: satisfying Boolean clauses

Literal: a variable or its negation. **Clause**: an OR of literals.

3SAT

AND of clauses, three literals per clause.
Can an assignment make **all clauses true**?

$$\phi = \underbrace{(x \vee y \vee z)}_{\text{clause 1}} \wedge \underbrace{(\neg x \vee y \vee z)}_{\text{clause 2}} \wedge \underbrace{(x \vee \neg y \vee \neg z)}_{\text{clause 3}}.$$

3SAT: satisfying Boolean clauses

Literal: a variable or its negation. **Clause**: an OR of literals.

3SAT

AND of clauses, three literals per clause.
Can an assignment make **all clauses true**?

$$\phi = \underbrace{(x \vee y \vee z)}_{\text{clause 1}} \wedge \underbrace{(\neg x \vee y \vee z)}_{\text{clause 2}} \wedge \underbrace{(x \vee \neg y \vee \neg z)}_{\text{clause 3}}.$$

One truth value per variable, shared across all clauses.

Checking a 3SAT assignment

$$\phi = (x \vee y \vee z) \wedge (\neg x \vee y \vee z) \wedge (x \vee \neg y \vee \neg z).$$

Try $x = \mathrm{T}$, $y = \mathrm{F}$, $z = \mathrm{T}$:

$$\underbrace{(\mathrm{T} \vee \mathrm{F} \vee \mathrm{T})}_{\mathrm{T}} \wedge \underbrace{(\mathrm{F} \vee \mathrm{F} \vee \mathrm{T})}_{\mathrm{T}} \wedge \underbrace{(\mathrm{T} \vee \mathrm{T} \vee \mathrm{F})}_{\mathrm{T}} = \mathrm{T}.$$

Checking a 3SAT assignment

$$\phi = (x \vee y \vee z) \wedge (\neg x \vee y \vee z) \wedge (x \vee \neg y \vee \neg z).$$

Try $x = \mathrm{T}$, $y = \mathrm{F}$, $z = \mathrm{T}$:

$$\underbrace{(\mathrm{T} \vee \mathrm{F} \vee \mathrm{T})}_{\mathrm{T}} \wedge \underbrace{(\mathrm{F} \vee \mathrm{F} \vee \mathrm{T})}_{\mathrm{T}} \wedge \underbrace{(\mathrm{T} \vee \mathrm{T} \vee \mathrm{F})}_{\mathrm{T}} = \mathrm{T}.$$

A short certificate

One bit per variable; evaluate every clause.

Polynomial-time check $\Longrightarrow$ **3SAT is in NP**.

Knapsack and 3SAT: checking

	Decision knapsack	3SAT
Certificate	Which items to take	Truth values of variables
Check	Budget and value bounds	Every clause is true
Exhaustive search	2^n subsets	2^m assignments

n: number of items. m: number of Boolean variables.

Knapsack and 3SAT: checking

	Decision knapsack	3SAT
Certificate	Which items to take	Truth values of variables
Check	Budget and value bounds	Every clause is true
Exhaustive search	2^n subsets	2^m assignments

n: number of items. m: number of Boolean variables.

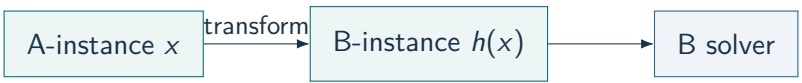
Easy to check. Easy to **decide**?

Reductions

Polynomial-time reduction $A \leq_p B$

Compute $h(x)$ in polynomial time, preserving the answer:

$$x \text{ is Yes for A} \iff h(x) \text{ is Yes for B.}$$

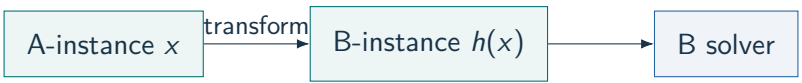


Reductions

Polynomial-time reduction $A \leq_p B$

Compute $h(x)$ in polynomial time, preserving the answer:

$$x \text{ is Yes for A} \iff h(x) \text{ is Yes for B.}$$



Polynomial-time B solver $\Longrightarrow$ polynomial-time A solver.

Subset sum reduces to knapsack

Subset sum

Positive integers $a_1, \ldots, a_n$; target t.
Does a subset sum to **exactly** t?

Subset sum reduces to knapsack

Subset sum

Positive integers $a_1, \ldots, a_n$; target t.
Does a subset sum to **exactly** t?

Make knapsack items $p_i = v_i = a_i$; set $B = T = t$.

$$\underbrace{\sum_{i \in S} a_i \leq t}_{\text{price bound}} \quad \text{and} \quad \underbrace{\sum_{i \in S} a_i \geq t}_{\text{value target}} \quad \Longleftrightarrow \quad \sum_{i \in S} a_i = t.$$

Subset sum reduces to knapsack

Subset sum

Positive integers $a_1, \ldots, a_n$; target t.
Does a subset sum to **exactly** t?

Make knapsack items $p_i = v_i = a_i$; set $B = T = t$.

$$\underbrace{\sum_{i \in S} a_i \leq t}_{\text{price bound}} \quad \text{and} \quad \underbrace{\sum_{i \in S} a_i \geq t}_{\text{value target}} \quad \Longleftrightarrow \quad \sum_{i \in S} a_i = t.$$

Example: $[1, 2, 3, 4]$, $t = 5 \quad \longrightarrow \quad$ take items 1 and 4.

NP-complete problems

MUST UNDERSTAND **NP-complete**

A decision problem B is NP-complete when:

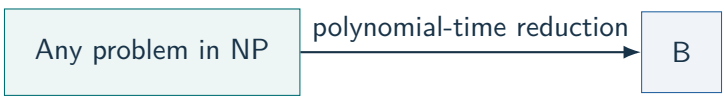
1. B belongs to NP.
2. **Every** problem in NP reduces to B in polynomial time.

NP-complete problems

MUST UNDERSTAND **NP-complete**

A decision problem B is NP-complete when:

1. B belongs to NP.
2. **Every** problem in NP reduces to B in polynomial time.



3SAT and decision knapsack are NP-complete

Theorem

3SAT and **decision 0/1 knapsack** are both NP-complete.

Knapsack: positive integer prices and values, in binary.

$(x \vee y \vee z) \wedge \cdots$

Choose truth values.



Choose treasures that fit.

Theorems stated without proof.

3SAT and decision knapsack are NP-complete

Theorem

3SAT and **decision 0/1 knapsack** are both NP-complete.

Knapsack: positive integer prices and values, in binary.

$(x \vee y \vee z) \wedge \cdots$

Choose truth values.

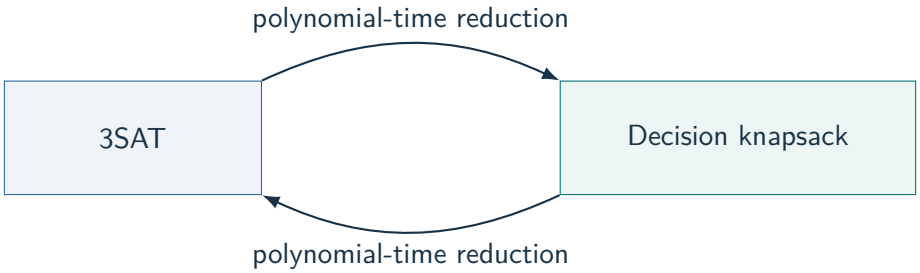


Choose treasures that fit.

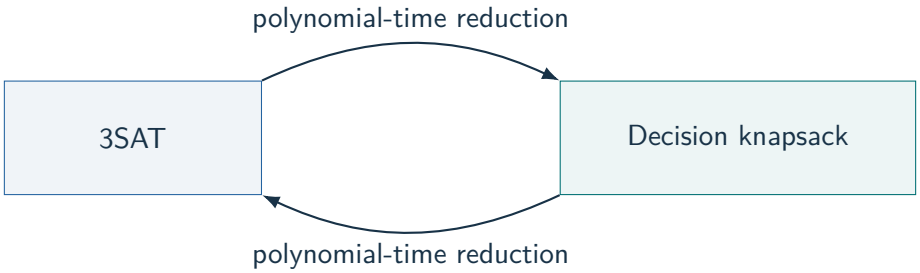
Every NP problem reduces to either one.

Theorems stated without proof.

Two-way translations



Two-way translations



Why both directions?

Each is in NP.
Each accepts reductions from every NP problem.

A solver for one solves all

$$\text{3SAT} \in \mathrm{P} \iff \text{Decision Knapsack} \in \mathrm{P} \iff \mathbf{P = NP}.$$

A solver for one solves all

$$\text{3SAT} \in \mathrm{P} \iff \text{Decision Knapsack} \in \mathrm{P} \iff \mathbf{P = NP}.$$

Suppose decision knapsack is in P

Any NP problem $\longrightarrow$ knapsack $\longrightarrow$ solve.

Both steps take polynomial time.

Approximation versus exact decision

Modified greedy guarantees

$$\text{Value}^{\text{Carol}} \leq \text{Best} \leq 2\text{Value}^{\text{Carol}}.$$



A

Price: 3

Value: 5



B

Price: 2

Value: 3



C

Price: 2

Value: 3

Approximation versus exact decision

Modified greedy guarantees

$$\text{Value}^{\text{Carol}} \leq \text{Best} \leq 2\text{Value}^{\text{Carol}}.$$



Budget 4; target 6

Returned value: 5. Thus $5 \leq \text{Best} \leq 10$.

Target 6? The bounds cannot decide.
Yet B + C gives 6: **Yes**.

Knapsack: three perspectives

Approach	Question	Result
Brute force	Find the optimum?	Exact answer; 2^n candidates
Modified greedy	Get good value quickly?	At least half of optimum
NP-completeness	Exact decision in polynomial time?	Would imply $\mathrm{P} = \mathrm{NP}$

Exit questions

1. Half-optimum guarantee; returned value 12. Bound Best.
2. One selection misses the target. Is the answer No?
3. Decision knapsack is in P. What follows for 3SAT and P vs NP?

Exit questions

1. Half-optimum guarantee; returned value 12. Bound Best.
2. One selection misses the target. Is the answer No?
3. Decision knapsack is in P. What follows for 3SAT and P vs NP?

$12 \leq \mathrm{Best} \leq 24$; no; 3SAT is in P and $\mathrm{P} = \mathrm{NP}$.

Reading

- Rosen 3.1.5: greedy algorithms.
- Rosen 3.3.4–3.3.5: brute force and computational complexity.
- Course note: knapsack approximation proof, certificates, and reductions.

Further reading: Cornell knapsack approximation notes;

UMass notes on subset sum and knapsack reductions.

Optional review: repeated squaring.

Optional: Repeated squaring

Repeated squaring: reduce arithmetic work

Compute $a^b \bmod m$, with integers $b \geq 0$ and $m \geq 2$.

Repeated multiplication

Multiply by a, b times, starting from 1.

Repeated squaring: reduce arithmetic work

Compute $a^b \bmod m$, with integers $b \geq 0$ and $m \geq 2$.

Repeated multiplication

Multiply by a, b times, starting from 1.

Use binary powers

For $b = 13 = 8 + 4 + 1$,

$$a^{13} = a^8 a^4 a.$$

Build powers by squaring.

Example: compute $3^{13} \bmod 7$

Exponent	Residue modulo 7	Calculation
1	3	Given
2	2	$3^2 \equiv 2$
4	4	$2^2 \equiv 4$
8	2	$4^2 \equiv 2$

Example: compute $3^{13} \bmod 7$

Exponent	Residue modulo 7	Calculation
1	3	Given
2	2	$3^2 \equiv 2$
4	4	$2^2 \equiv 4$
8	2	$4^2 \equiv 2$

$$13 = 8 + 4 + 1 \quad \Longrightarrow \quad 3^{13} \equiv 2 \cdot 4 \cdot 3 \equiv \mathbf{3} \pmod 7.$$

Repeated squaring: one loop

Pseudocode

$r \leftarrow 1$; $s \leftarrow a \bmod m$; $e \leftarrow b$
while $e > 0$:
  if e is odd: $r \leftarrow rs \bmod m$
  $s \leftarrow s^2 \bmod m$
  $e \leftarrow \lfloor e/2 \rfloor$
return r

Repeated squaring: one loop

Pseudocode

$r \leftarrow 1$; $s \leftarrow a \bmod m$; $e \leftarrow b$
while $e > 0$:
  if e is odd: $r \leftarrow rs \bmod m$
  $s \leftarrow s^2 \bmod m$
  $e \leftarrow \lfloor e/2 \rfloor$
return r

Preserve

$$rs^e \equiv a^b \pmod{m}.$$

If $e = 2q + d$, $d \in \{0, 1\}$:

$$(rs^d)(s^2)^q = rs^e.$$

At $e = 0$, the answer is r.

Repeated squaring: count the work

For $b \geq 1$, halve the exponent each time:

$$b,\ \lfloor b/2 \rfloor,\ \lfloor b/4 \rfloor, \ldots, 1.$$

Iterations

$$\lfloor \log_2 b \rfloor + 1$$

Per iteration

At most two modular multiplications.

Repeated squaring: count the work

For $b \geq 1$, halve the exponent each time:

$$b,\ \lfloor b/2 \rfloor,\ \lfloor b/4 \rfloor, \ldots, 1.$$

Iterations

$$\lfloor \log_2 b \rfloor + 1$$

Per iteration

At most two modular multiplications.

$O(\log b)$ **modular multiplications**.

Bit cost depends on the modulus length.